

جامعة دمشق

كلية الهندسة المعلوماتية — قسم هندسة البرمجيات

تقرير مشروع مادة البرمجة المتوازية

الفصل الدراسي 2026

محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

(High-Performance E-Commerce Backend Engine)

ShopFlow

التقنيات المستخدمة: • Spring Boot 3.4.3 • Java 17 • PostgreSQL • Redis

Nginx • Docker Compose • Prometheus • Grafana

فهرس المحتويات

1. مقدمة

2. المتطلب الأول — حماية البيانات المشتركة من التضارب

2.1 القفل المتفائل (Optimistic Locking)

2.2 القفل المتشائم (Pessimistic Locking)

2.3 نمط الاستراتيجية (Strategy Pattern)

3. المتطلب الثاني — إدارة الموارد الحاسوبية

3.1 إدارة اتصالات قاعدة البيانات (HikariCP)

3.2 إدارة اتصالات Redis (Lettuce Pool)

3.3 إدارة مجمعات الخيوط (Thread Pool Executors)

4. المتطلب الثالث — المعالجة غير المتزامنة

4.1 نمط الحدث/المستمع (Event/Listener Pattern)

4.2 إرسال البريد الإلكتروني بشكل غير متزامن

4.3 إبطال الرموز عبر Redis

5. المتطلب الرابع — معالجة البيانات الضخمة على دفعات

5.1 المهمة المجدولة (Scheduled Job)

5.2 المعالجة على شكل دفعات (Chunk Processing)

6. المتطلب الخامس — توزيع الأحمال

6.1 البنية التحتية (Docker Compose + Nginx)

6.2 استراتيجية التوزيع (Round-Robin via DNS)

6.3 تسجيل الطلبات عبر AOP

6.4 المراقبة باستخدام Prometheus و Grafana

7. ملخص الملفات والتقنيات

1. مقدمة

يهدف مشروع **ShopFlow** إلى بناء نظام خلفي (Backend) متكامل لمنظومة تجارة إلكترونية، مع التركيز على المتطلبات غير الوظيفية المتعلقة بالبرمجة المتوازية والأداء العالي. يُغطي هذا التقرير المتطلبات الخمسة الأولى من متطلبات المشروع، مع شرح مفصّل للحلول المعمارية والكود المصدري المستخدم لتحقيق كل متطلب.

تم بناء النظام باستخدام إطار عمل **Spring Boot 3.4.3** مع **Java 17**، ويعتمد على **PostgreSQL 16** كقاعدة بيانات رئيسية، و **Redis 7** لإدارة الرموز والتخزين المؤقت، و **Nginx** كموزع أحمال، و **Docker Compose** لتنسيق الحاويات. كما تم دمج **Prometheus** و **Grafana** لمراقبة الأداء في الزمن الحقيقي.

2. المتطلب الأول — حماية البيانات المشتركة من التضارب

(Concurrent Access & Data Integrity)

يتطلب النظام آلية تسمح لعدة مستخدمين بتعديل نفس المورد (مثل كمية المنتج في المخزون) دون حدوث تضارب أو فقدان بيانات. تم تطبيق نهج مزدوج يجمع بين القفل المتفائل للعمليات العادية والقفل المتشائم للعمليات ذات التضارب العالي (مثل التخفيضات السريعة).

2.1 القفل المتفائل (Optimistic Locking)

يعتمد القفل المتفائل على افتراض أن التضارب نادر الحدوث. يُضاف حقل `Version@` لكيان المنتج، يزداد تلقائياً مع كل عملية تحديث. إذا حاول خيطان تعديل نفس السجل في الوقت ذاته، يفشل الثاني بخطأ `OptimisticLockException`.

كيان المنتج مع حقل الإصدار:

```
src/main/java/.../product/entity/Product.java
```

```
@Entity
@Table(name = "products")
public class Product extends BaseEntity {

    private String name;
    private int stockQuantity;
    private BigDecimal price;

    @Version
    @Column(nullable = false)
    @Builder.Default
    private Long version = 0L;
}
```

عند تنفيذ عملية التحديث، يُؤلّد JPA استعلام SQL يتضمن شرط

الإصدار:

```
UPDATE products
SET stock_quantity = ?, version = version + 1
WHERE id = ? AND version = ?
```

إذا لم يتطابق الإصدار (أي أن خطأ آخر عدّل السجل)، لا يُحدّث أي سطر ويُطرح استثناء. تتعامل استراتيجية الشراء المتفائلة مع هذا الاستثناء عبر إعادة المحاولة:

استراتيجية الشراء بالقفل المتفائل:

```
src/main/java/.../product/strategy/OptimisticPurchaseStrategy.java
```

```

@Component("optimisticPurchase")
public class OptimisticPurchaseStrategy implements
PurchaseStrategy {

    private static final int MAX_RETRIES = 3;

    @Override
    @Transactional
    public Product purchase(UUID productId, int quantity) {
        int attempt = 0;
        while (attempt < MAX_RETRIES) {
            try {
                return attemptPurchase(productId, quantity);
            } catch (ObjectOptimisticLockingFailureException
ex) {
                attempt++;
                log.warn("Optimistic lock conflict - attempt
{}/{}", attempt, MAX_RETRIES);
            }
        }
        throw new ConflictException("Could not complete
purchase after " + MAX_RETRIES + " retries");
    }

    private Product attemptPurchase(UUID productId, int
quantity) {
        Product product =
productRepository.findById(productId)
                .orElseThrow(() -> new
ResourceNotFoundException("Product not found"));
        if (product.getStockQuantity() < quantity) {
            throw new InsufficientStockException("Not enough
stock");
        }
        product.setStockQuantity(product.getStockQuantity()
- quantity);
        return productRepository.save(product);
    }
}

```

2.2 القفل المتشائم (Pessimistic Locking)

في حالة التخفيضات السريعة (Flash Sales)، يكون التضارب شبه مؤكد مع عشرات المستخدمين يتنافسون على كمية محدودة. هنا يُستخدم القفل المتشائم الذي يقفل السجل على مستوى قاعدة البيانات فوراً باستخدام

.SELECT ... FOR UPDATE

الاستعلام مع القفل المتشائم:

```
src/main/java/.../flashsale/repository/FlashSaleRepository.java
```

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT f FROM FlashSale f WHERE f.id = :id")
Optional<FlashSale> findByIdWithLock(@Param("id") UUID id);
```

يُترجم هذا إلى استعلام SQL:

```
SELECT * FROM flash_sales WHERE id = ? FOR UPDATE
```

أي خيط آخر يحاول قراءة نفس السجل سيُحظر تلقائياً حتى ينتهي الخيط الأول من المعاملة (COMMIT أو ROLLBACK).

استراتيجية الشراء بالقفل المتشائم:

```
src/main/java/.../product/strategy/PessimisticPurchaseStrategy.java
```

```
@Component("pessimisticPurchase")
public class PessimisticPurchaseStrategy implements
PurchaseStrategy {

    @Override
    @Transactional
    public Product purchase(UUID flashSaleId, int quantity)
    {
        FlashSale flashSale =
flashSaleRepository.findByIdWithLock(flashSaleId)
                .orElseThrow(() -> new
ResourceNotFoundException("Flash sale not found"));

        if (flashSale.getRemainingQuantity() < quantity) {
            throw new InsufficientStockException("Flash sale
sold out");
        }

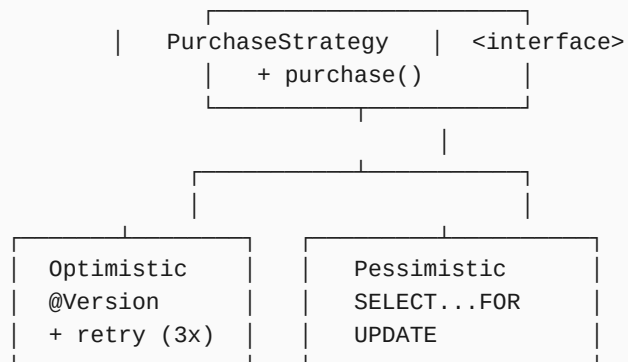
        flashSale.setRemainingQuantity(flashSale.getRemainingQuantit
y() - quantity);
        flashSaleRepository.save(flashSale);
        return flashSale.getProduct();
    }
}
```

2.3 نمط الاستراتيجية (Strategy Pattern)

تم تطبيق نمط التصميم Strategy لاختيار آلية القفل المناسبة حسب نوع الطلب. في خدمة الطلبات، يتم تفويض اختيار القفل إلى الاستراتيجية المناسبة:

```
// الشراء العادي → قفل متفائل
Product product =
optimisticStrategy.purchase(itemReq.getProductId(),
itemReq.getQuantity());

// تخفيضات سريعة → قفل متشائم
Product product = pessimisticStrategy.purchase(flashSaleId,
quantity);
```



3. المتطلب الثاني — إدارة الموارد الحاسوبية

(Resource Management & Capacity Control)

يتطلب النظام التحكم في عدد العمليات المتوازية لمنع استنزاف الموارد (انهيار) أو تقليلها بشكل يبطئ الاستجابة. تم تطبيق هذا المتطلب على ثلاثة مستويات: اتصالات قاعدة البيانات، اتصالات Redis، ومجموعات الخيوط.

3.1 إدارة اتصالات قاعدة البيانات (HikariCP)

HikariCP هو مجمع اتصالات عالي الأداء يأتي مدمجاً مع Spring Boot. تم ضبطه للسماح بحد أقصى 10 اتصالات متزامنة لقاعدة البيانات:

src/main/resources/application.yml

```
spring:
  datasource:
    hikari:
      pool-name: HikariPool-AuthDB
      maximum-pool-size: 10      # الحد الأقصى للاتصالات
      minimum-idle: 5           # الحد الأدنى للاتصالات
      idle-timeout: 300000       # دقائق - إغلاق الاتصالات 5
      connection-timeout: 30000 # ثانية - فشل سريع بدلاً 30
      max-lifetime: 1800000      # دقيقة - إعادة تدوير 30
      leak-detection-threshold: 60000 # تحذير إذا بقي اتصال مفتوحاً أكثر من 60 ثانية
```

المعامل	القيمة	الهدف
maximum-pool-size	10	منع إغراق PostgreSQL بعدد كبير من الاتصالات
minimum-idle	5	ضمان جاهزية 5 اتصالات دائماً لتقليل زمن الاستجابة
connection-timeout	30 ثانية	فشل سريع بدلاً من حجز الخيط إلى ما لا نهاية
leak-detection-threshold	60 ثانية	كشف تسريب الاتصالات (نسيان إغلاقها)

3.2 إدارة اتصالات Redis (Lettuce Pool)

يُستخدم Redis لإبطال رموز JWT وتخزين البيانات المؤقتة. تم ضبط مجمع Lettuce لتحديد عدد الاتصالات المتزامنة:

```
spring:
  data:
    redis:
      lettuce:
        pool:
          max-active: 10      # الحد الأقصى للاتصالات المتزامنة
          max-idle: 5         # اتصالات خاملة جاهزة
          min-idle: 2         # اتصالات دافئة دائماً
          max-wait: 2000ms    # فشل بعد ثانيتين إذا لم
                              # يتوفر اتصال
```

3.3 إدارة مجمعات الخيوط (Thread Pool Executors)

تم تعريف مجمعي خيوط منفصلين لعزل الموارد بين المهام المختلفة ومنع تأثير نوع من المهام على نوع آخر:

```
src/main/java/.../shared/config/AsyncConfig.java
```

```

@Configuration
@EnableAsync
@EnableScheduling
public class AsyncConfig {

    // مجمع خيوط لمعالجة أحداث الطلبات (فواتير، إشعارات)
    @Bean(name = "orderExecutor")
    public Executor orderExecutor() {
        ThreadPoolTaskExecutor executor = new
ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);           // خيوط أساسية 5
        executor.setMaxPoolSize(20);          // يتوسع حتى 20
        // خيط تحت الضغط
        executor.setQueueCapacity(100);        // طابور يستوعب
        // 100 مهمة
        executor.setThreadNamePrefix("order-async-");
        executor.setRejectedExecutionHandler(
            new ThreadPoolExecutor.CallerRunsPolicy() //
تدهور أنيق
        );
        executor.initialize();
        return executor;
    }

    // مجمع خيوط مخصص للمهام الدفعية (معزول عن حركة الطلبات)
    @Bean(name = "batchExecutor")
    public Executor batchExecutor() {
        ThreadPoolTaskExecutor executor = new
ThreadPoolTaskExecutor();
        executor.setCorePoolSize(2);
        executor.setMaxPoolSize(5);
        executor.setQueueCapacity(10);
        executor.setThreadNamePrefix("batch-job-");
        executor.setRejectedExecutionHandler(
            new ThreadPoolExecutor.CallerRunsPolicy()
        );
        executor.initialize();
        return executor;
    }
}

```

ملاحظة معمارية — CallerRunsPolicy:

عند امتلاء الطابور والوصول للحد الأقصى من الخيوط، بدلاً من رفض المهمة (وفقدانها)، يقوم الخيط المُستدعي بتنفيذها بنفسه. هذا يُبطئ قبول المهام الجديدة بشكل طبيعي (Back-pressure) دون فقدان أي بيانات.

Application Thread Pools	
orderExecutor Core: 5 Max: 20 Queue: 100	batchExecutor Core: 2 Max: 5 Queue: 10
Invoice generation Email sending Notifications	Daily sales report Token cleanup

4. المتطلب الثالث — المعالجة غير المتزامنة

(Asynchronous Queues)

يتطلب نقل المهام التي لا يحتاج المستخدم لانتظارها (مثل إصدار الفواتير أو إرسال الإشعارات) خارج المسار الرئيسي للطلب. بذلك يحصل المستخدم على استجابة فورية بينما تُنفَّذ المهام الثقيلة في الخلفية.

4.1 نمط الحدث/المستمع (Event/Listener Pattern)

عند إتمام طلب شراء، يُنشر حدث `OrderPlacedEvent` عبر نظام أحداث Spring. يلتقطه مستمع غير متزامن يعمل في خيط منفصل:

نشر الحدث عند إتمام الطلب:

src/main/java/.../order/service/OrderService.java

```
@Transactional
public Order placeOrder(UUID userId, PlaceOrderRequest
request) {
    // ... إنشاء الطلب وتحديث المخزون ...
    order = orderRepository.save(order);

    // نشر الحدث - المستمع يعالج الفاتورة بشكل غير متزامن
    eventPublisher.publishEvent(new OrderPlacedEvent(this,
order));
    log.info("Order placed - async invoice triggered");

    return order; // الاستجابة تُرسل فوراً قبل إنشاء الفاتورة
}
```

المستمع غير المتزامن:

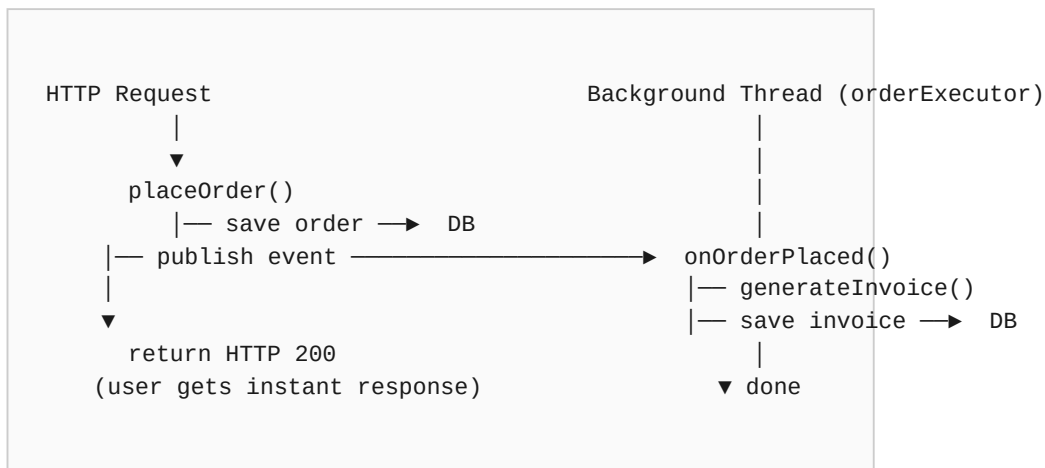
src/main/java/.../order/event/OrderEventListener.java

```

@Component
public class OrderEventListener {

    @Async("orderExecutor") // يعمل في مجمع خيوط الطلبات
    @EventListener
    public void onOrderPlaced(OrderPlacedEvent event) {
        Order order = event.getOrder();
        log.info("[ASYNC] Handling OrderPlacedEvent in
thread {}",
                Thread.currentThread().getName());
        try {
            invoiceService.generateInvoice(order);
            log.info("[ASYNC] Invoice generated for order
{}", order.getId());
        } catch (Exception ex) {
            log.error("[ASYNC] Failed to generate invoice
for order {}", order.getId());
        }
    }
}

```



4.2 إرسال البريد الإلكتروني بشكل غير متزامن

جميع عمليات إرسال البريد الإلكتروني (التحقق من الحساب، إعادة تعيين كلمة المرور) تتم بشكل غير متزامن حتى لا يُحظر الخيط الرئيسي بانتظار خادم SMTP:

```
src/main/java/.../auth/service/EmailService.java
```

```

@Service
public class EmailService {

    @Async    // يعمل في خيط منفصل
    public void sendVerificationEmail(String to, String
token) {
        String verificationUrl = baseUrl +
"/api/auth/verify-email?token=" + token;
        sendHtmlEmail(to, "Verify your email address",
body);
    }

    @Async
    public void sendPasswordResetEmail(String to, String
token) {
        String resetUrl = baseUrl + "/api/auth/reset-
password?token=" + token;
        sendHtmlEmail(to, "Reset your password", body);
    }
}

```

4.3 إبطال الرموز عبر Redis

عند تسجيل الخروج، يتم إدراج رمز JWT في القائمة السوداء عبر Redis مع مدة انتهاء تلقائية (TTL)، مما يمنع إعادة استخدام الرمز:

```

public void blacklist(String jti, long remainingMs) {
    String key = "jwt:blacklist:" + jti;
    redisTemplate.opsForValue().set(key, "1",
Duration.ofMillis(remainingMs));
    // Redis المفتاح تلقائياً بعد انتهاء المدة
}

```

5. المتطلب الرابع — معالجة البيانات الضخمة على دفعات

(Batch Processing)

يتطلب برمجة وظيفة خلفية تقوم بجرد المبيعات اليومية ومعالجتها كدفعات (Chunks) لزيادة سرعة الأداء. تم تحقيق ذلك عبر مهمة مجدولة تستخدم الترقيم الصفحي (Pagination) لتقسيم البيانات.

5.1 المهمة المجدولة (Scheduled Job)

يتم تشغيل تقرير المبيعات اليومية تلقائياً عند منتصف الليل بتوقيت UTC، في مجمع خيوط مخصص ومعزول عن حركة الطلبات:

```
src/main/java/.../report/scheduler/DailySalesJob.java
```

```
@Component
public class DailySalesJob {

    @Async("batchExecutor") // 5 أساسي، 2 مخصص
    @Scheduled(cron = "0 0 0 * * *", zone = "UTC") // كل يوم عند منتصف الليل
    public void runDailyReport() {
        LocalDate yesterday = LocalDate.now().minusDays(1);
        log.info("[BATCH] Starting daily sales report for {} in thread {}",
            yesterday,
            Thread.currentThread().getName());
        reportService.generateReportForDate(yesterday);
    }
}
```

5.2 المعالجة على شكل دفعات (Chunk Processing)

بدلاً من تحميل جميع الطلبات في الذاكرة دفعة واحدة (مما قد يسبب OutOfMemoryError)، تتم المعالجة على دفعات بحجم 500 سجل باستخدام Pageable:

```
src/main/java/.../report/service/DailySalesReportService.java
```

```

@Service
public class DailySalesReportService {

    private static final int CHUNK_SIZE = 500;

    @Transactional
    public void generateReportForDate(LocalDate date) {
        // التحقق من عدم وجود تقرير مُسبق (idempotency)
        if (reportRepository.existsByReportDate(date)) {
            log.info("Report for {} already exists - skipping", date);
            return;
        }

        DailySalesReport report = DailySalesReport.builder()
            .reportDate(date)
            .status(ReportStatus.PROCESSING)
            .build();
        report = reportRepository.save(report);

        int page = 0;
        int totalOrders = 0;
        BigDecimal totalRevenue = BigDecimal.ZERO;
        Page<Order> chunk;

        try {
            do {
                // سجل في كل مرة CHUNK_SIZE جلب
                chunk =
                orderRepository.findByStatusAndCreatedAtBetween(
                    OrderStatus.CONFIRMED, from, to,
                    PageRequest.of(page, CHUNK_SIZE)
                );

                for (Order order : chunk.getContent()) {
                    totalRevenue =
                    totalRevenue.add(order.getTotalAmount());
                }
                totalOrders += chunk.getNumberOfElements();

                log.info("Batch chunk {} processed - {} orders", page, chunk.getNumberOfElements());
                page++;

            } while (chunk.hasNext());

            report.setTotalOrders(totalOrders);
            report.setTotalRevenue(totalRevenue);
            report.setStatus(ReportStatus.COMPLETED);
            reportRepository.save(report);

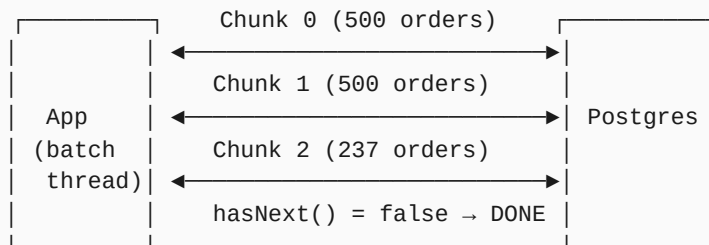
        } catch (Exception ex) {
            report.setStatus(ReportStatus.FAILED);
            reportRepository.save(report);
            log.error("Daily report for {} failed: {}",

```

```

date, ex.getMessage());
    }
}
}

```



Memory at any point: max 500 Order objects (not 1237)

ملاحظة — تنظيف الرموز المنتهية:

بالإضافة لتقرير المبيعات، توجد مهمة مجدولة أخرى تعمل كل ساعة لحذف رموز التحديث المنتهية والملغاة من قاعدة البيانات:

```

@Scheduled(fixedRate = 3600000) // كل ساعة
@Transactional
public void cleanupStaleTokens() {

    refreshTokenRepository.deleteExpiredTokens(Instant.now());

    refreshTokenRepository.deleteRevokedTokensBefore(now.minus(24, ChronoUnit.HOURS));
    log.info("Token cleanup completed");
}

```

6. المتطلب الخامس — توزيع الأحمال

(Load Distribution)

يتطلب محاكاة توزيع الطلبات على أكثر من خادم مع توضيح وتبرير استراتيجية التوزيع المستخدمة. تم تحقيق ذلك عبر Nginx كموزع أحمال أمام نسختين من تطبيق Spring Boot، مُدارة بواسطة Docker Compose.

6.1 البنية التحتية (Docker Compose + Nginx)

يتم تشغيل نسختين من التطبيق عبر أمر التوسيع في Docker Compose:

```
docker compose up --scale app=2
```

هذا يُنشئ حاويتين متطابقتين (app-1 و app-2) خلف شبكة Docker واحدة. يتعامل Nginx مع جميع الطلبات الواردة على المنفذ 80 ويوزعها بين النسختين:

(المقتطفات الأساسية): docker-compose.yml

```
services:
  app:
    image: shopflow:latest
    # ضروري للتوسيع - container_name
    # يتعامل مع الوصول الخارجي Nginx - ports
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - app
```

6.2 استراتيجية التوزيع (Round-Robin via Docker DNS)

تم اعتماد استراتيجية **Round-Robin** (التناوب الدوري) لتوزيع الطلبات بالتساوي بين النسختين. يعتمد التطبيق على نظام DNS الداخلي لـ Docker الذي يُعيد عناوين IP مختلفة عند كل استعلام لاسم الخدمة app.

النقطة الحرجة: يجب استخدام متغير في proxy_pass لإجبار Nginx على إعادة الاستعلام عن DNS في كل طلب (بدون ذلك يُخزن Nginx العنوان مرة واحدة عند الإقلاع ويُرسَل جميع الطلبات لنسخة واحدة):

nginx/nginx.conf

```
server {
    listen 80;
    server_name localhost;

    # إعادة الاستعلام كل 5 ثواني - Docker الداخلي لـ DNS مُحلل
    resolver 127.0.0.11 valid=5s ipv6=off;

    # حجب نقاط الوصول الداخلية من الخارج
    location /actuator {
        deny all;
        return 403;
    }

    location / {
        # في كل طلب DNS المتغير يُجبر إعادة استعلام
        set $backend http://app:8080;
        proxy_pass $backend;

        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

لماذا Round-Robin وليس استراتيجية أخرى؟

بما أن نسختي التطبيق متطابقتان في الموارد والإعدادات، فإن التوزيع بالتساوي هو الخيار الأمثل. استراتيجيات مثل Least Connections أو IP Hash تُضيف تعقيداً دون فائدة عندما تكون الخوادم متماثلة.

6.3 تسجيل الطلبات عبر AOP

لإثبات أن التوزيع يعمل فعلياً، تم تطبيق مفهوم البرمجة الموجهة للجوانب (AOP) لاعتراض كل طلب HTTP وتسجيل مُعرّف النسخة التي عالجه:

```
src/main/java/.../shared/config/RequestLoggingAspect.java
```

```
@Aspect
@Component
@Slf4j
public class RequestLoggingAspect {

    @Value("${HOSTNAME:single}")
    private String instanceId;

    @Around("@within(org.springframework.web.bind.annotation.RestController)")
    public Object logRequest(ProceedingJoinPoint joinPoint)
        throws Throwable {
        HttpServletRequest request =
            ((ServletRequestAttributes)
                RequestContextHolder.getRequestAttributes()).getRequest();

        String method = request.getMethod();
        String uri = request.getRequestURI();

        log.info("[INSTANCE: {}] --> {} {} (handler: {})",
            instanceId, method, uri,
            joinPoint.getSignature().toShortString());

        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed();
        long duration = System.currentTimeMillis() - start;

        log.info("[INSTANCE: {}] <-- {} {} completed in {}ms",
            instanceId, method, uri, duration);

        return result;
    }
}
```

عند تشغيل app logs docker compose وإرسال عدة طلبات، تظهر السجلات بالتناوب بين النسختين:

```
app-1 | [INSTANCE: aa087c423ab6] --> GET /api/products
completed in 13ms
app-2 | [INSTANCE: 32433b18c8ce] --> GET /api/products
completed in 13ms
app-1 | [INSTANCE: aa087c423ab6] --> GET /api/products
completed in 2ms
app-2 | [INSTANCE: 32433b18c8ce] --> GET /api/products
completed in 3ms
```

6.4 المراقبة باستخدام Prometheus و Grafana

لمراقبة أداء الطلبات ومدتها في الزمن الحقيقي، تم دمج **Prometheus** لجمع المقاييس و **Grafana** لعرضها بصرياً. يستخدم التطبيق **Spring Boot** **Actuator** مع **Micrometer** لتصدير المقاييس بتنسيق Prometheus:

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
  metrics:
    tags:
      application: shopflow
    distribution:
      percentiles-histogram:
        http.server.requests: true
      percentiles:
        http.server.requests: 0.5,0.95,0.99
```

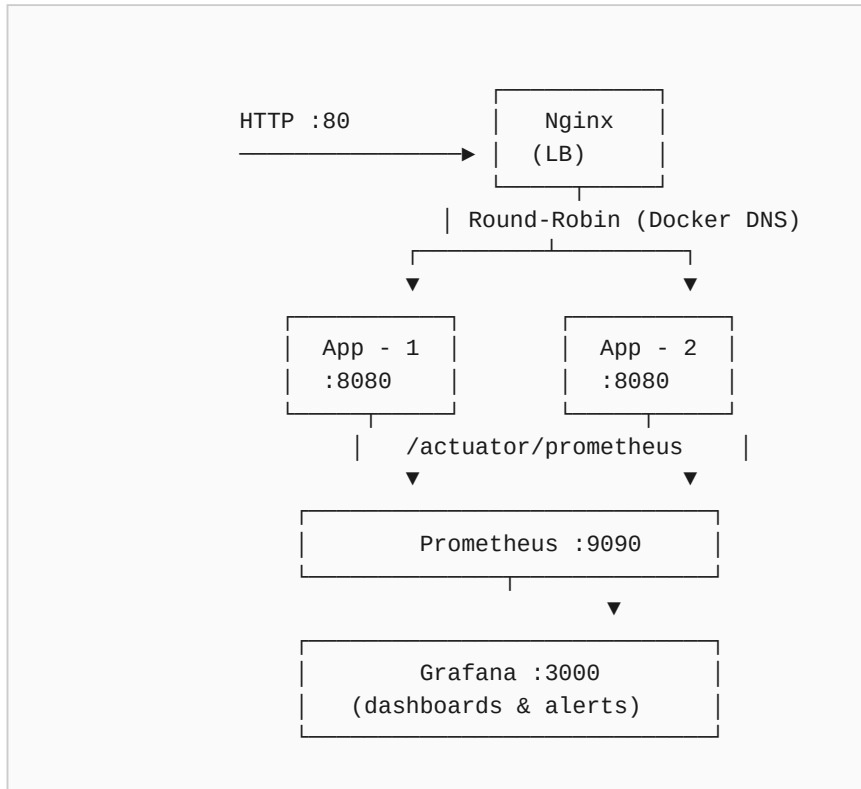
يقوم Prometheus بجمع المقاييس من كلتا النسختين تلقائياً عبر اكتشاف DNS:

```
scrape_configs:
  - job_name: 'shopflow'
    metrics_path: '/actuator/prometheus'
    dns_sd_configs:
      - names: ['app']
        type: 'A'
        port: 8080
```

تم إعداد لوحة مراقبة Grafana مُسبقة التهيئة تعرض:

- عدد الطلبات المعالجة لكل نسخة (لإثبات توزيع الحمل)

- جدول تفصيلي بكل مسار: المدة المتوسطة، المدة القصوى، عدد الطلبات
- حالة الخيوط (نشطة، خاملة، محجوبة) كأرقام مباشرة
- حالة مجمع اتصالات قاعدة البيانات (HikariCP)
- معدل الطلبات في الثانية لكل نسخة ورموز حالة HTTP



7. ملخص الملفات والتقنيات

المتطلب	التقنية / النمط	الملفات الرئيسية
1. حماية البيانات من التضارب	Optimistic Locking (@Version) Pessimistic Locking (SELECT FOR UPDATE) Strategy Pattern	Product.java OptimisticPurchaseStrategy.java PessimisticPurchaseStrategy.java FlashSaleRepository.java
2. إدارة الموارد الحاسوبية	HikariCP Connection Pool Lettuce Redis Pool ThreadPoolTaskExecutor CallerRunsPolicy	application.yml AsyncConfig.java
3. المعالجة غير المتزامنة	Async + @EventListener@ Spring ApplicationEvent Observer Pattern	OrderPlacedEvent.java OrderEventListener.java InvoiceService.java EmailService.java
4. معالجة البيانات على دفعات	Scheduled (cron)@ Pageable Chunk Processing Idempotent execution	DailySalesJob.java DailySalesReportService.java TokenCleanupScheduler.java
5. توزيع الأحمال	Nginx Round-Robin Docker Compose scaling AOP Request Logging Prometheus + Grafana	docker-compose.yml nginx/nginx.conf RequestLoggingAspect.java */monitoring